Sécurité logicielle (HAI821I)

Master Informatique
Département Informatique
Faculté des Sciences de Montpellier
Université de Montpellier

Examen du 27 juin 2023

Tous les documents de cours sont autorisés. L'examen dure 2h. Le barème est donné à titre indicatif. Le sujet comporte 2 pages et il y a 2 exercices.

Exercice 1 (10 pts)

On considère l'ensemble des expressions de notre langage impératif vu en cours, qui est défini de la manière suivante :

$$\begin{array}{rcl} e & ::= & n \mid x \mid e_1 + e_2 \mid e_1 - e_2 \mid e_1 \times e_2 \mid e_1/e_2 \\ & \mid & \text{true} \mid \text{false} \mid \text{not}(e) \mid e \text{ and } e \mid e \text{ or } e \\ & \mid & e = e \mid e \mathrel{!=} e \mid e < e \mid e \leq e \mid e \geq e \mid e > e \end{array}$$

où $n \in \mathbb{Z}$ et $x \in \mathbb{V}$ (ensemble de noms de variables).

On souhaite rajouter un système de types sur ce langage afin de restreindre l'ensemble des expressions évaluables. Par exemple, nous ne souhaitons pas évaluer l'expression $1 + \text{true}$.

L'ensemble des types est défini comme suit :

$$\tau \quad ::= \quad \text{int} \mid \text{bool}$$

On définit la notion d'environnement de typage Γ comme étant une liste de couples (x_i, τ_i), où x_i est une variable et τ_i un type. La relation de typage $\Gamma \vdash e : \tau$, qui se lit « l'expression e est de type τ dans l'environnement Γ », est définie au moyen des règles suivantes :

$$\frac{}{\Gamma \vdash n : \text{int}}\,\tau_{\mathbb{Z}} \qquad \frac{(x,\tau) \in \Gamma}{\Gamma \vdash x : \tau}\,\tau_{\mathbb{V}} \qquad \frac{\Gamma \vdash e_1 : \text{int} \qquad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{int}}\,\tau_{\text{op}},\ \text{avec op} \in \{+, -, \times, /\}$$

$$\frac{}{\Gamma \vdash \text{true} : \text{bool}}\,\tau_{\text{true}} \qquad \frac{}{\Gamma \vdash \text{false} : \text{bool}}\,\tau_{\text{false}}$$

$$\frac{\Gamma \vdash e : \text{bool}}{\Gamma \vdash \text{not}(e) : \text{bool}}\,\tau_{\text{not}} \qquad \frac{\Gamma \vdash e_1 : \text{bool} \qquad \Gamma \vdash e_2 : \text{bool}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{bool}}\,\tau_{\text{op}},\ \text{avec op} \in \{\text{and}, \text{or}\}$$

$$\frac{\Gamma \vdash e_1 : \text{int} \qquad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 \text{ op } e_2 : \text{bool}}\,\tau_{\text{op}},\ \text{avec op} \in \{=, \mathrel{!=}, <, \leq, \geq, >\}$$

On remarquera, au passage, que nous souhaitons uniquement comparer des expressions arithmétiques entre elles (pas des expressions booléennes).

1. Typer les expressions suivantes (vous devrez construire des arbres de typage pour ces expressions en utilisant les règles introduites ci-dessus) :

(a) $((2 \times 3) + x) > y$, dans l'environnement $\Gamma = (x, \text{int}), (y, \text{bool})$.

(b) $\text{not}(x = 1)$ and $(x! = y)$, dans l'environnement $\Gamma = (x, \text{int}), (y, \text{int})$.

2. Dans ce système de types, la vérification de type est décidable, c'est-à-dire la possibilité de vérifier qu'une expression est d'un type donné dans un environnement de typage donné. Écrire la fonction de vérification de type. Elle prendra en paramètre une expression e, un environnement Γ et un type τ et rendra $\top$ (*vrai*) si la relation $\Gamma \vdash e : \tau$ est vérifiée et $\bot$ (*faux*) sinon.

3. Démontrer la correction du typage vis-à-vis de la sémantique à grands pas. Cette propriété peut être énoncée comme suit :

   Pour tout environnement de typage Γ, pour toute expression e et pour tout type τ tels que $\Gamma \vdash e : \tau$ alors pour tout environnement d'exécution E tel $\mathsf{dom}(\Gamma) = \mathsf{dom}(E)$:

   — Si $\tau = \text{int}$ alors il existe $n \in \mathbb{Z}$ tel que $E \vdash e \rightsquigarrow n$.

   — Si $\tau = \text{bool}$ alors il existe $b \in \mathbb{B} = \{\top, \bot\}$ tel que $E \vdash e \rightsquigarrow b$.

   Cette propriété se démontre par simple induction structurelle sur l'expression e. Attention à bien préciser et utiliser les hypothèses d'induction dans les cas inductifs. Par ailleurs, il vous faudra également « inverser » la relation de typage $\Gamma \vdash e : \tau$ pour déterminer le type τ suivant la forme de e.

Exercice 2 (10 pts)

On souhaite rajouter l'instruction "do ... while" à notre ensemble d'instructions avec la syntaxe suivante :

$$i \quad ::= \quad \ldots \mid \text{do } i \text{ while } e.$$

Intuitivement, l'exécution d'une instruction "do i while c" consiste à exécuter l'instruction i, puis à évaluer la condition c. Si la condition est vraie, on exécute à nouveau l'instruction "do i while c". Si la condition est fausse, on s'arrête (on sort de la boucle).

1. Donner les règles d'exécution de l'instruction "do i while c" dans la sémantique à grands pas et dans la sémantique à petits pas.

2. Exécuter l'instruction suivante dans les deux sémantiques à grands pas et à petits pas :

   do $i := i + 1$ while $i < 2$, dans l'environnement $E = (i, 0)$.

3. Démontrer que les deux instructions "do i while c" et "i ;while c do i" sont équivalentes dans la sémantique à petits pas.